



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

Chandigarh University Uttar Pradesh

School of Computer Science & Engineering

Semester III

Python Programming

Course Code: 25 CSH-214

Faculty Name: Prateek Singhal

Python

Course Objectives

Students learn basic syntax, data types, control structures, functions, and the use of standard libraries such as NumPy and math

Students will be able to solve real-time problem-solving.

By the end of the course, students develop the ability to write efficient programs, evaluate their correctness and performance.



CO-PO Mapping

COs/ Pos	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	2	2	2	-	-	-	-	-	-	2	3	2
CO2	3	3	2	2	3	-	-	-	-	-	-	2	3	3
CO3	3	3	2	2	3	-	-	-	-	-	-	2	3	3
CO4	3	3	2	2	3	-	-	-	-	-	-	2	3	3
CO5	3	3	3	3	2	3	2	2	3	2	2	3	3	3

UNIT 1

Topics to be Covered

- Conditional Execution
- Repeated Execution
- Reuse (Functions)
- Types of Errors in Programs
- Debugging a Program

The Four Pillars of Pythonic Architecture

1. Control

Directing the flow. Advanced conditionals, pattern matching, and iteration.

```
match status:
    case 'active':
        process_data()

for item in items:
    if item.is_valid:
```

Pattern Matching

Conditional Logic

Iterative Logic (with Filter)

2. Structure

Packaging logic. Function blueprints, arguments, and scoping rules.

```
def build_structure(blueprint_id,
                  *args, **kwargs):

    # Global vs. Local Scope
    local_config = setup_scope(blueprint_id)

    components = assemble_components(
        *args, **kwargs)
```

Function Blueprint

Scoping Rules

Arguments (Positional & Keyword)

3. Refine

Eliminating bloat. The DRY principle and clean code extraction.

```
Before
task1.process();
task1.validate();

task2.process();
task2.validate();

After
# Clean Extraction
def execute_task(task):
    task.process();
    task.validate();

execute_task(task1);
execute_task(task2);
```

Repeated Code (Bloat)

The DRY Principle

Refactored Usage

4. Protect

Armoring against chaos. Exception handling and call stack tracebacks.

```
try:
    dangerous_operation()
except ValueError as e:

    # Log & Recover
    log_error(e);
    recover_state()
```

Exception Handling Start

Catch Specific Error

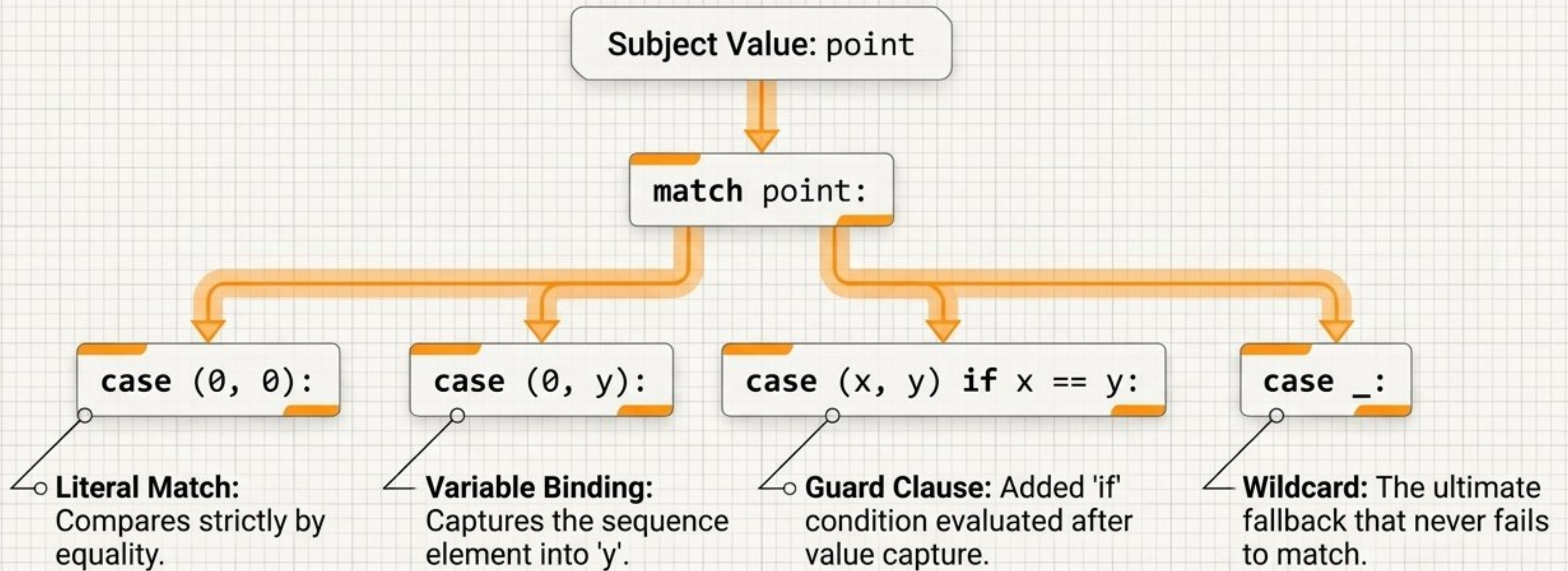
Traceback (most recent call last):
File "main.py", line 42, in <module>
File "core.py", line 10, in dangerous_operation

ValueError: Invalid input received.

Call Stack Tracebacks

Error Details

Directing Execution via Pattern Matching

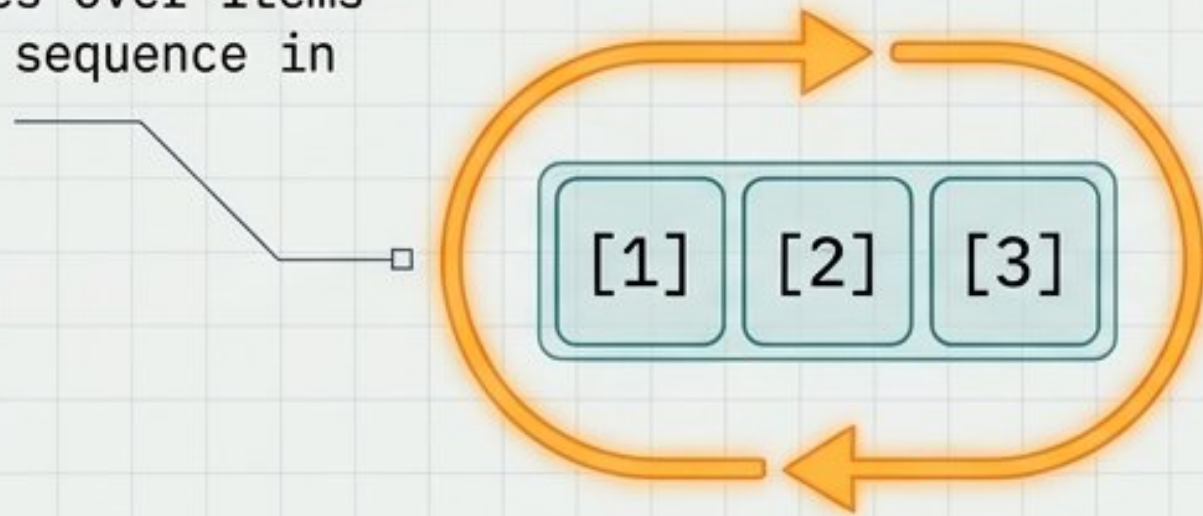


Takeaway: 'match' doesn't just evaluate truth—it structurally unpacks sequences, binds variables, and filters via guards in a single pass.

Mastering the Iteration: The Looping Matrix

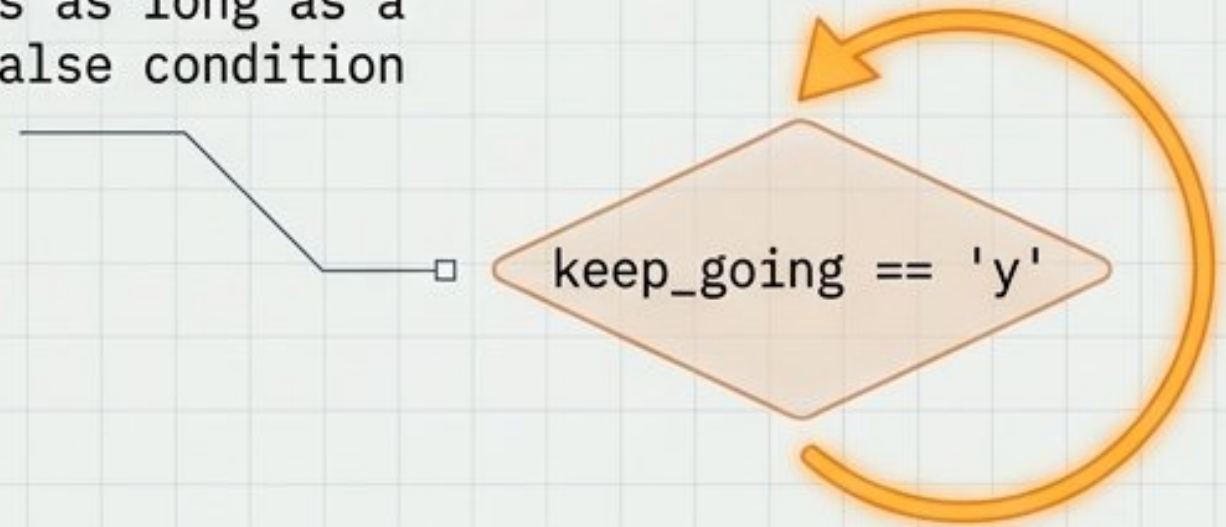
Count-Controlled (for)

Iterates over items of any sequence in order.



Condition-Controlled (while)

Repeats as long as a true/false condition holds.



Flow Modifiers

break

Exits the innermost loop entirely.

continue

Skips immediately to the next iteration.

else

Python specific. Executes ONLY if the loop finishes without hitting a break.

The Structural Blueprint of a Function

Instantiates the function object in memory.

Optional metadata annotations for type hinting.

```
def compute_velocity(distance: float, time: float) -> float:
    """Calculates velocity. Time must be > 0."""
    velocity = distance / time
    return velocity
```

The Docstring. The first statement, stripped of indentation by the parser, used for documentation.

Passes an expression back to the caller (defaults to None if omitted).

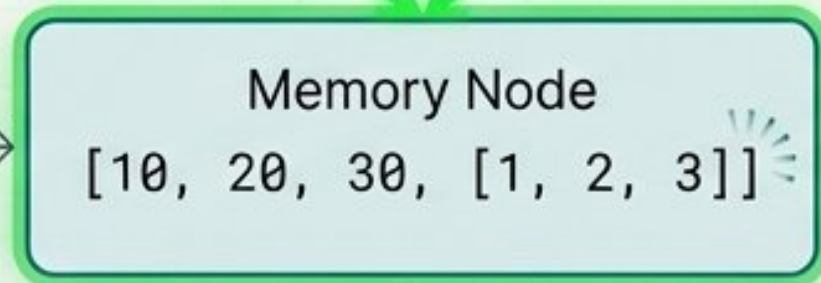
Scope, Memory, and Passing by Reference

Mutating the Reference

```
mylist.append([1,2,3])
```

Global Variable

Local Function Variable

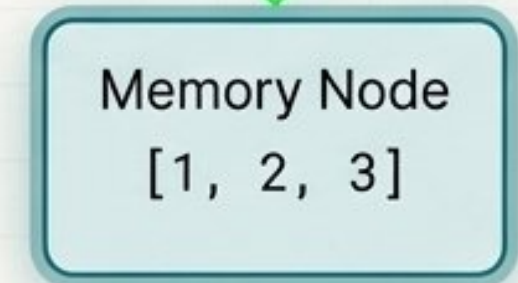
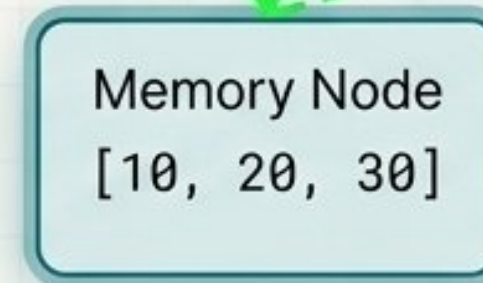


Overwriting the Reference

```
mylist = [1,2,3]
```

Global Variable

Local Function Variable



Global list remains unaffected.

The Argument Architecture

Required (Positional)

Passed in exact positional order. Missing arguments yield a `TypeError`.

```
→ def printme(str):  
→ printme() / TypeError — Missing required  
                        argument: "str"
```

Keyword

Caller identifies arguments by name, allowing skipped or out-of-order placement.

```
→ printinfo(age=50, name='miki')
```

Matched by name "age" → ← Matched by name "name"

Default

Function assumes a pre-defined value if none is provided.

```
→ def printinfo(name, age=35):
```

Warning: Evaluated only once at definition, dangerous for mutable objects!

Default value for 'age' if not passed

Variable-Length

Scoops up arbitrary remaining inputs.

- `*args` creates a tuple.
- `**kwargs` creates a dictionary.

```
→ def printinfo(arg1, *vartuple):
```

Collects extra positional arguments into a tuple named 'vartuple'

Precision Parameter Control

Parameter Funnel

```
def func(pos_only, /, standard, *, kwd_only):
```

Positional-Only. These arguments MUST be positional. Prevents API breaks if parameter names change in the future.

Standard arguments. Can be positional or keyword.

Keyword-Only. Enforces explicit readability for arguments where the name carries critical meaning.

The Anonymous Function (lambda)

sum = **lambda** **arg1, arg2:** **arg1 + arg2**

The **keyword** creating the anonymous function object.

The **parameter list** (can take any number).

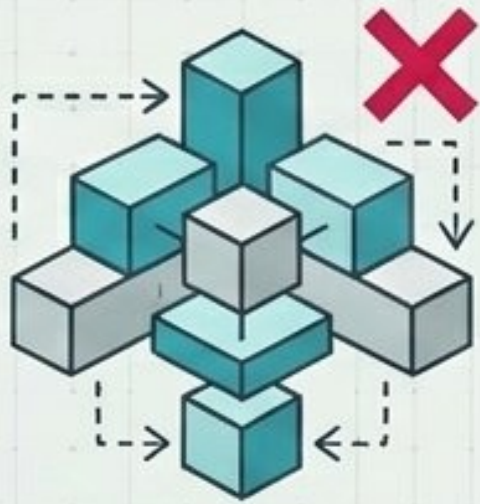
The single expression.

Key Rules

- Restricted to a single expression (no commands or multiple statements).
- Returns one value implicitly (no 'return' keyword needed).
- Maintains its own local namespace.
- Often used to pass small functions as arguments (e.g., `list.sort(key=lambda x: ...)`).

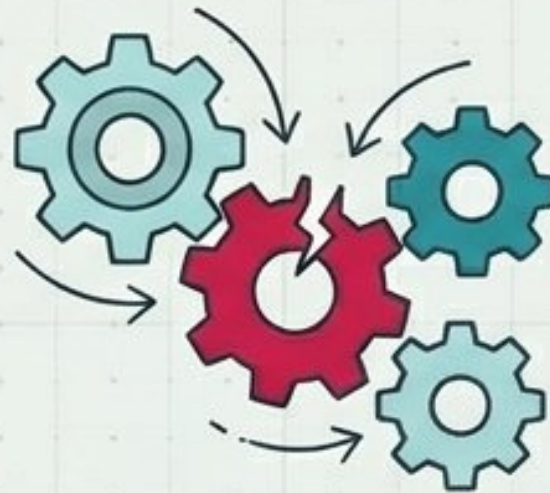
The DRY Principle: Why Redundancy Fails

Code Bloat



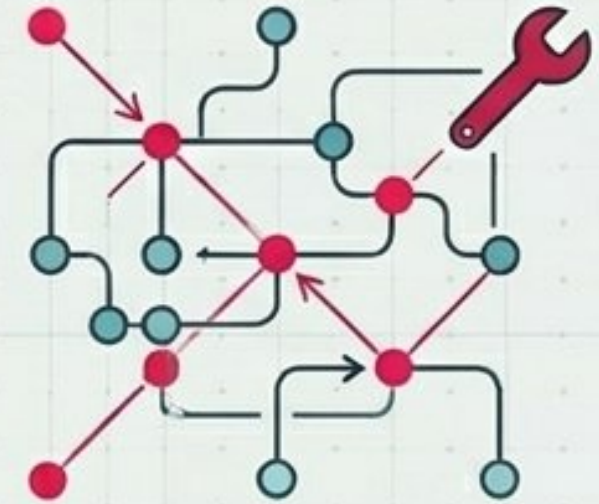
Unnecessarily increases codebase size, making navigation difficult and increasing the error surface area.

Risk of Inconsistencies



Scattered logic becomes out of sync during updates, silently introducing hidden bugs into the system.

Maintenance Challenges



Fixing a single bug requires modifications in multiple disparate locations, taxing developer bandwidth.

Takeaway: Clean code is not just about aesthetics; it is a defensive strategy for long-term maintainability.

The Refactoring Toolkit

1. Extract Function

Before

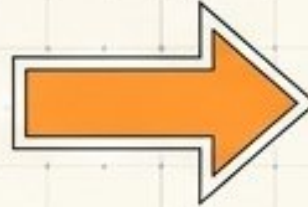
Standard Total

```
subtotal = item1.price +  
item2.price;  
tax = subtotal * 0.08;  
total = subtotal + tax;
```

Heavy bloated

Express Total

```
subtotal = item1.price +  
item2.price;  
tax = subtotal * 0.08;  
total = subtotal + tax +  
15.00;
```



After

```
calculate_items_total()
```

Encapsulate repeated logic
into a reusable function.

2. Extract Variable

Before

```
final_price = price * 0.90 * 0.95;
```

Cryptic Decimals



After

```
loyalty_adjusted_price = price * LOYALTY_DISCOUNT;
```

Meaningful Name

3. Replace Temp with Function

Before

```
new_customer = (order_count == 0);  
if (new_customer) {  
    apply_discount();  
}  
if (new_customer) {  
    send_welcome_email();  
}
```

new_customer



After

```
if (is_new_customer()) {  
    apply_discount();  
}  
if (is_new_customer()) {  
    send_welcome_email();  
}
```

Compute on Demand

Anticipating Chaos: Anatomy of a Crash

IDE console

```
> x = float('12a')  
Traceback (most recent call last):  
  File "prompt.py", line 13, in <module>  
    x = float(result)  
ValueError: could not convert string to float: '12a'
```

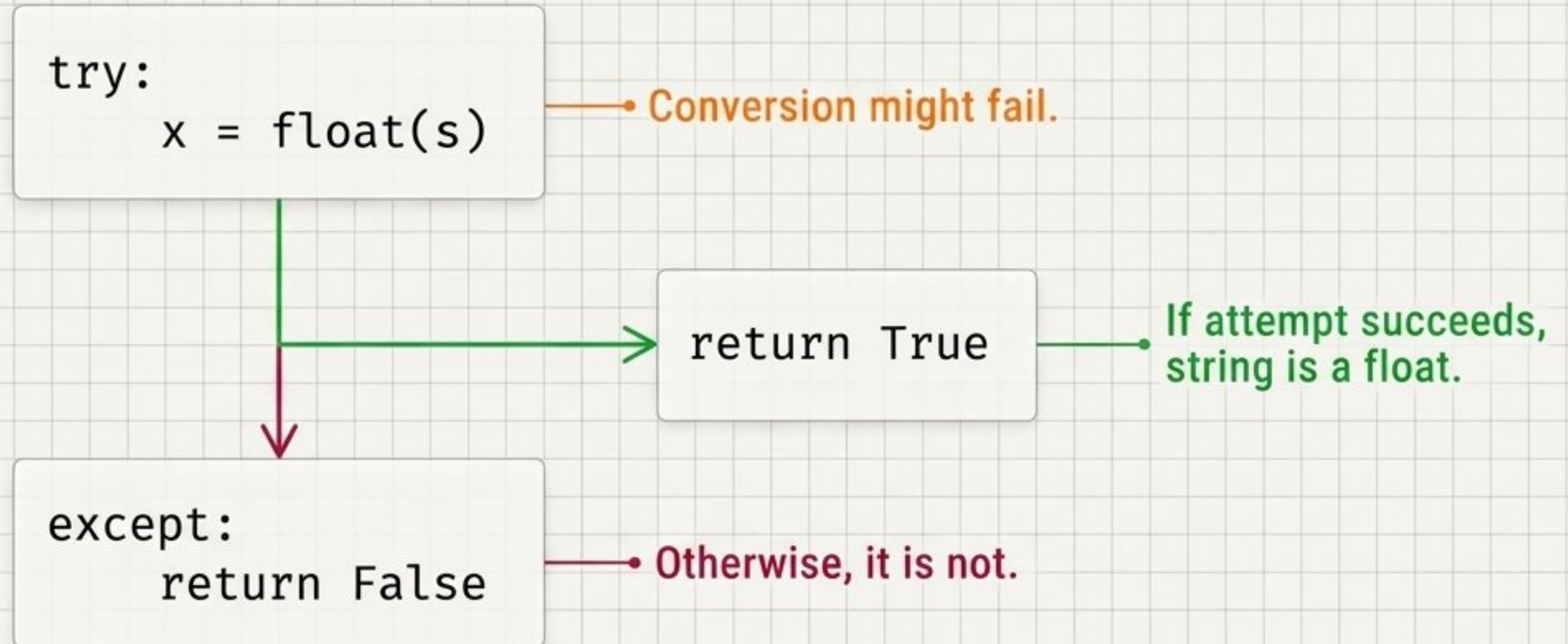
The Call Stack history.

The exact location of
the failure.

The specific Exception
type and description.

Unpredictable inputs bypass standard conditionals. We need a safety net.

The Execution Safety Net



Key Concept: Unlike if-else, the system *always* enters the 'try' block but might abort halfway through if an error is raised.

Design Philosophy: Permission vs. Forgiveness

LBYL (Look Before You Leap)	EAFP (Easier to Ask for Forgiveness)
Mechanism <pre>if property_holds: proceed() else: handle_failure()</pre> Uses conditionals (if/else) to check if a property holds before proceeding.	Mechanism <pre>try: proceed() except: handle_failure()</pre> Uses try/except. Assumes the property holds, runs the code, and catches the failure if it doesn't.
Philosophy Asking for Permission.	Philosophy Asking for Forgiveness.

Python actively prefers EAFP. Exceptions in Python are highly optimized and 'cheap' compared to other languages, making this a legitimate flow control strategy—though it should only be used when it significantly simplifies code.